

XGOUD

Entwicklerdokumentation

Self-built WordPress-Blocktheme `ekinese`

Edelmetall-Ankauf · staging.xgoud.nl · Stand: Juni 2026

1. Überblick & Architektur

XGOUD ist ein **selbstgebautes WordPress-Blocktheme** (Full-Site-Editing) mit dem Slug `ekinese`. Die gesamte Geschäftslogik – Rechner, Termine, Fleet/Fahrer-App, Auktionen, Marktplatz, Portfolio, Loyalty, News, SEO, Zahlungen, AI – ist **im Theme** implementiert. Es werden **keine Plugins** verwendet; einzige externe Bausteine sind Redis (Object-Cache), Cloudflare (CDN/Edge-Cache) und optionale APIs (Anthropic, Mollie, SMS, goldapi.io, OpenRouteService, OAuth-Provider).

- **Repo-Root = Theme-Root.** `style.css` deklariert „Theme Name: Ekinese“. Läuft in `wp-content/themes/ekinese/`.
- **Kein Build-Schritt nötig.** Kein Composer/npm im Produktiv-Theme. Dev-Hilfsdateien (`build-*.js`, `*-preview.html`, `test-suite.html`) gehören *nicht* ins Release-ZIP.
- **95 Module** unter `inc/`, geladen über `functions.php` (Reihenfolge ist relevant – Helfer vor Verwenden).
- **64 REST-Endpunkte** unter Namespace `ekinese/v1`, **35 CPTs**, **38 dynamische Blocks**.

Brand-Regeln: Rot `#AE1E1E` primär, Gold `#D0AC4B` als Highlight, Dunkel `#161412`, **border-radius 0**. Diese gelten in Light- und Dark-Mode.

2. Verzeichnisstruktur

```
ekinese/
├─ style.css           Theme-Header + Basis-Styles
├─ theme.json          FSE-Settings (Farben, Typo, Spacing)
├─ functions.php       lädt alle inc/-Module (require_once, Reihenfolge!)
├─ inc/               95 PHP-Module (Geschäftslogik)
├─ assets/
│  ├─ css/            19 Stylesheets (blueprint, theme, dark, ...)
│  └─ js/             43 Skripte (calculator, account, driver-app, ...)
├─ patterns/          Block-Patterns (page-home, hero-calculator, ...)
├─ parts/             header.html, footer.html (FSE template parts)
├─ templates/         FSE-Templates
├─ styles/            theme.json-Style-Varianten
└─ data/              products.json, listings.csv, i18n.json, ...
```

3. Modulübersicht (inc/)

Geladen in dieser Reihenfolge in `functions.php`. Gruppiert nach Funktion:

Bereich	Module
Setup & Basis	<code>setup</code> , <code>patterns</code> , <code>block-styles</code> , <code>taxonomies</code> , <code>blueprint-sections</code> , <code>installer</code> , <code>admin-menu</code>
Katalog & Preise	<code>products</code> , <code>watches</code> , <code>gemstones</code> , <code>diamond</code> , <code>verkopen</code> , <code>price-chart</code> , <code>price-tabs</code> , <code>price-forecast</code> , <code>ratios</code> , <code>spot-prices</code> , <code>hallmark</code> , <code>lexicon</code>

Rechner & Formulare	forms , calc-save , scheduling , booking-slots
Konto & Auth	account , registration , social-login , portfolio , loyalty , rewards , lottery , year-review , savings-goal
Operation	tickets , chat , questions , pickup , kyc , notifications , notify , mail-flows , smtp
Fleet / Fahrer	fleet , driver , verify
Handel	auctions , market , marketplace , payments , inventory , partners , business-pro , business-portal
Content & Community	news , google-news , moments , surveys , trust , compare , faq , kennisbank , charity , offices , city-pages , newsletter
SEO / Analytics	seo , seo-index , seo-editor , llms , analytics , heatmap , stats , social , social-insights , ads , integrations
AI & Dashboard	admin-dashboard , assistant , ai-assistant , photo-appraisal , optimizer
Performance / PWA	perf , perf-images , pwa , widget , push , live-ticker , floating-contact , sticky-cta
Recht / Sicherheit	legal , consent , security , accounting , hr , appraisal-cert

4. Datenmodell – Custom Post Types

35 CPTs. Identität von Kund:innen ist **immer die E-Mail** (kein WP-User nötig). Auswahl:

CPT	Zweck	Wichtige Meta
xg_appointment	Termine/Ankäufe	email, date, time, service, status, proforma, vcode, verified_at
xg_account	Profil (E-Mail-Login)	email, name, phone, pass_hash, phone_verified
xg_product / xg_watch / xg_gemstone	Katalog	metal, category, weight, carat, ...
xg_auction / xg_market	Auktion / Marktplatz	status, ends, paid, charity_pct
xg_moment	UGC-Wall	email, type, display_name, likes, points_awarded
xg_survey / xg_survey_response	Umfragen	questions(JSON), survey_status, points / survey, email, answers
xg_driver / xg_shift / xg_route / xg_expense	Fleet	token, stops(JSON), km, ...
xg_ticket / xg_question / xg_pickup / xg_kyc	Service/Compliance	email, status, reference
xg_reward / xg_holding / xg_price_alert	Loyalty/Portfolio	points, metal, target
xg_pushsub	Web-Push-Abos	endpoint, keys

5. REST-API (ekinese/v1)

64 Endpunkte. Authentifizierung kundenseitig über **Account-Token** (signiert, 24 h), adminseitig über `current_user_can('manage_options')` bzw. Nonces. Auswahl nach Bereich:

Bereich	Endpunkte (Methode)
Konto/Auth	<code>account/login</code> (POST, magic-link), <code>account/data</code> (GET, token), <code>account/register</code> , <code>account/login-password</code> , <code>account/phone/start</code> , <code>account/phone/verify</code> (POST)
Preise	<code>prices</code> (GET, €/g + Trend + Sparkline)
Community	<code>moments</code> (GET), <code>moments/submit</code> , <code>moments/like</code> (POST), <code>surveys/{id}</code> (GET), <code>surveys/submit</code> (POST)
Handel	<code>market/submit</code> , <code>market/remove</code> , <code>pay/start</code> , <code>pay/webhook</code> (Mollie)
Fleet	<code>fleet/...</code> (route-token-basiert), <code>verify/check</code> , <code>fleet/verify</code>
Rewards	<code>reward/share</code> , <code>reward/referral</code> , <code>price-alert</code> , <code>calc-save</code>

Konvention für neue Routen: in `rest_api_init` registrieren, `permission_callback` immer setzen (öffentliche Routen `__return_true` + interne Validierung), Eingaben `sanitize_*`, Honeypot-Feld `website` + Rate-Limit via Transient.

6. Blocks (dynamische Blöcke)

38 Blocks via `register_block_type('ekinese/...', ['render_callback' => ...])`. Vorbild: `inc/price-chart.php`, `inc/widget.php`.

Kritische Konvention: Ein dynamischer Block, der *innerhalb* eines `<!-- wp:html -->`-Blocks platziert wird, rendert als **inert HTML-Kommentar** (er läuft nicht!). Dynamische Blocks müssen **Top-Level-Geschwister** sein, gewrappt in `<!-- wp:group {"className":"xg-blueprint"} -->` für CSS-Scoping.

```
<!-- wp:group {"className":"xg-blueprint"} -->
<div class="wp-block-group xg-blueprint">
  <!-- wp:ekinese/moments {"limit":8} /-->
</div>
<!-- /wp:group -->
```

Asset-Gating: JS/CSS eines Blocks nur laden, wenn der Block vorhanden ist – `if ( is_singular() && has_block('ekinese/...') ) in wp_enqueue_scripts`.

7. Konto- & Auth-System

Vier gleichwertige Login-Wege, die alle im selben **Account-Token** enden
(`ekinese_account_make_token($email)` / `ekinese_account_verify_token($token)`):

1. **Magic-Link** (`account.php`): E-Mail → signierter Link.
2. **E-Mail + Passwort** (`registration.php`): Hash via `wp_hash_password` .
3. **Telefon-OTP**: 6-stelliger SMS-Code (MessageBird/Twilio).
4. **Social** (`social-login.php`): Google/Facebook/Apple OAuth → `/xg-oauth/{provider}/callback/` .

Das Token gibt Zugriff auf `account/data` ; dort hängen sich Module über den Filter `ekinese_account_data($data,$email)` ein (Portfolio, Loyalty, Momente, Surveys, ...). Frontend: `assets/js/account.js` (Widget-Dashboard).

8. Styling-System

Datei	Zweck
<code>style.css</code> / <code>theme.json</code>	Theme-Header, FSE-Tokens (Farben/Typo/Spacing)
<code>theme.css</code>	Day/Dark-Switch, Brand-Variablen (<code>--red</code> , <code>--gold</code> , <code>--ink</code>)
<code>blueprint.css</code>	Landingpage-Sektionen (Hero, Cards, Grids, Steps) – scoped auf <code>.xg-blueprint</code>
<code>dark.css</code>	Zentrale Dark-Schicht, nur unter <code>html[data-theme="dark"]</code> , als letztes geladen
<code>header-footer.css</code>	Topbar, Mega-Menü/Wizard, Footer, Mobile-Menü
<code>locale.css</code>	Mijn-XGOUD-Dashboard (Widgets, Tiers, Timeline, Login-Tabs)
weitere	<code>calculator</code> , <code>charity</code> , <code>chat</code> , <code>offices</code> , <code>auctions</code> , <code>moments</code> , <code>surveys</code> , <code>driver-app</code> , <code>widget</code> , <code>assistant</code> , <code>consent</code> , <code>newsletter</code> , <code>main</code>

Scoping: `.xg-grid-*` , `.xg-container` , `.xg-section` wirken nur unter Vorfahre `.xg-blueprint` .

Reveal: `html.xg-js .xg-reveal { opacity:0 }` – ohne JS voll sichtbar (Fail-Safe). **Dark Mode:** jede Komponente bringt `html[data-theme="dark"]` -Regeln mit; `dark.css` füllt Restlücken.

9. JavaScript-Assets (Auswahl von 43)

Datei	Zweck
<code>theme.js</code>	Day/Dark (folgt OS), Reveal-Observer + Fail-Safe; im <code><head></code> (kein FOUC)
<code>calculator.js</code>	Wertrechner (Metall/Diamant/Edelstein/Uhr) + Charity + Checkout
<code>account.js</code>	Mijn XGOUD: Login-Tabs + verschiebbares Widget-Dashboard
<code>header-footer.js</code>	Mega-Menü/Wizard (Desktop + Mobile), Ticker, AI-Suche
<code>driver-app.js</code>	Fahrer-PWA (Schicht, km, Stops/ETA, OCR, KYC, GPS, Scanner)
<code>moments.js</code> / <code>surveys.js</code>	UGC-Wall / Umfrage-Frontend
<code>seo-editor.js</code>	SEO/GEO-Analyzer (RankMath-Stil) im Block-Editor
<code>pwa.js</code> / Service Worker	Installierbar, Web-Push (VAPID, aes128gcm – ohne Library)

10. Optionen / API-Schlüssel (Referenz)

Option	Zweck
<code>xg_anthropic_key</code> , <code>xg_anthropic_model</code>	Claude (AI-Assistent, Optimizer, Vision). Default-Modell <code>claude-haiku-4-5</code>
<code>xg_perf</code>	XG-caching-Konfiguration (Array)
<code>xg_mollie_key</code>	Mollie-Zahlungen (Webhook <code>/pay/webhook</code>)
<code>xg_sms_provider</code> , <code>xg_sms_key</code> , <code>xg_sms_twilio_*</code> , <code>xg_sms_from</code>	Telefon-OTP
<code>xg_si_*</code>	Social-Insights-Tokens (yt/fb/ig/li/tt/tw)
<code>xg_oauth_*</code> , <code>xg_oauth_apple_secret</code>	Social-Login (Google/Facebook/Apple)
<code>xg_spot_api_key</code> , <code>xg_metals_spot</code>	goldapi.io Spot-Preise → <code>ekinese_metal_spot()</code>
<code>xg_ors_key</code>	OpenRouteService (ETA Fleet)
<code>xg_idex_key/secret</code>	IDEX-Diamantpreise
<code>xg_ga4_id</code> , <code>xg_gtag_id</code> , <code>xg_gsc_verify</code> , <code>xg_bing_verify</code> , <code>xg_fb_pixel</code>	Analytics/Verifikation (Consent-gated)
<code>xg_smtp_*</code>	SMTP-Versand
<code>xg_vapid</code>	Web-Push-Schlüsselpaar
<code>xg_legal_kvkk/btw/iban/opkoper</code>	Rechtsangaben
<code>xg_recaptcha_*</code>	reCAPTCHA v3 (Anti-Spam)

11. Konventionen – Neues hinzufügen

Neues Modul

1. `inc/mein-modul.php` anlegen, mit `if ( ! defined('ABSPATH') ) exit; .`
2. `require_once get_theme_file_path('inc/mein-modul.php');` in `functions.php` (Reihenfolge beachten – nach den genutzten Helfern).
3. `php -l` (Lint) + Kollisions-Check (keine doppelten Funktionsnamen/CPT-Keys/REST-Routen/Submenu-Slugs).

Neue Admin-Seite

```
add_action('admin_menu', function(){
    add_submenu_page('xgoud', 'Titel', 'Titel', 'manage_options', 'xg-slug', 'render_fn');
});
```

Gruppierung im Menü: Slug-Fragment in das passende Array in `inc/admin-menu.php` aufnehmen (z. B. „Einstellungen“).

Statistik-/AI-Anbindung

- Daily Task: Filter `ekinese_daily_tasks_extra` → `['key','label','count','link']`.
- AI-Kontext: Filter `ekinese_dashboard_ai_context_lines` → Strings anhängen.
- Konto-Daten: Filter `ekinese_account_data($data,$email)`.
- Punkte: `ekinese_award_points($email,$points,$reason,$ref)` ; Werte über `ekinese_reward_rules`.

12. Build & Deploy

Kein Build-Tooling im Release. Sauberes ZIP aus dem committeten Stand:

```
git archive --format=zip --prefix=ekinese/ -o ekinese-stage.zip HEAD
zip -d ekinese-stage.zip 'ekinese/build-*.js' 'ekinese/demo-content.js' \
    'ekinese/*-preview.html' 'ekinese/*-demo.html' 'ekinese/test-suite.html'
```

Upload über *Design* → *Themes* → *Theme hochladen* → *ersetzen* (oder FTP nach `wp-content/themes/ekinese/`). **Branch:** `claude/optimistic-hopper-9gy6xn`. Danach Pflicht-Schritte siehe Betriebsanleitung (XGOUD-Setup, Permalinks, Cache).

13. Performance & SEO/GEO

XG caching (`perf.php` / `perf-images.php`) ersetzt FlyingPress: CSS/JS-Minify, Defer/Delay-JS, Lazyload, Lite-YouTube, Fonts-Swap, Hover-Preload, HTML-Minify, WP-Bloat-Cleanup, DB-Cron, Cloudflare-Edge-Cache (Header + API-Purge), WebP/AVIF bei Upload. Logged-in-User werden bewusst nicht optimiert.

SEO/GEO (`seo.php` / `seo-editor.php`): Focus-Keyword + Live-Analyzer (zwei Scores), Server-Scorer (`_xg_seo_score` / `_xg_geo_score`) mit Spalte in Pages/Posts. Sitemaps: `/sitemap.xml` , `/news-sitemap.xml` , `/llms.txt` , IndexNow.